

Mutual Exclusion and Synchronisation in FreeRTOS

PGSCE – Programação Avançada de Sistemas Embebidos

António Barros

Introduction to FreeRTOS Mutexes

The Need for Mutual Exclusion

- **Shared Resources:** Multiple tasks often access the same hardware (UART, I2C) or global variables.
- **Race Conditions:** Simultaneous access can lead to data corruption or unpredictable behaviour.
- **The Solution:** Mutex (Mutual Exclusion)
 - ▶ Acts as a *"token"* or *"key"*.
 - ▶ Only the task holding the token can access the resource.
 - ▶ Other tasks must wait (block) until the token is returned.

Mutex (Mutual Exclusion)

- **Concept of Ownership:** Acts as a token or unique key to access a shared resource.
- **Ownership:** Only the task that *took* the Mutex may *give* (release) it.
- **Protection Mechanism:** Specifically designed to isolate critical sections.
- **Priority Inheritance:** FreeRTOS automatically implements this protocol to prevent unbounded priority inversion.

Analogy

A dressing room key: the person who enters holds the key, and only the person inside can unlock the door upon leaving.

Binary Semaphore

- **Concept of Signalling:** Acts as a flag or signal to indicate that an event has occurred.
- **No Ownership:** Any task (or interrupt) can "give" the semaphore, and another task can "take" it.
- **States:** Has only two states: 0 (empty/unavailable) or 1 (available).
- **Typical Use:** Synchronisation between interrupts (ISRs) and tasks (Deferred Processing).

Analogy

A traffic light: it signals when you may proceed, but the signal does not "belong" to any individual vehicle.

Mutex vs Binary Semaphore

Feature	Binary Semaphore	Mutex
Main Usage	Signalling / Event Sync	Resource Protection
Ownership	No ownership (Any task can "give")	Yes (Only owner can "give")
Initial State	Usually 0 (Empty)	Always 1 (Available)
Priority Inheritance	Not Available	Mandatory Mechanism

A Mutex is conceptually a binary semaphore with a **priority inheritance** mechanism.

Create Mutex: xSemaphoreCreateMutex()

The xSemaphoreCreateMutex() function is used to create a new mutex-type semaphore.

```
#include "semphr.h"
```

```
SemaphoreHandle_t xSemaphoreCreateMutex(void);
```

- **Return Value:** Returns a handle to the created mutex, or NULL if there is insufficient heap memory.
- **Initial State:** The mutex is created in an "available" state, meaning it can be immediately taken by the first task that calls xSemaphoreTake().
- **Memory:** This function uses the FreeRTOS heap to allocate the memory required for the semaphore's control structure.

Create Mutex: xSemaphoreCreateMutex()

To use a mutex, you must first create it and store the handle.

```
#include "FreeRTOS.h"
#include "semphr.h"

SemaphoreHandle_t xMutex;

void vSetup(void) {
    xMutex = xSemaphoreCreateMutex();    // Create the mutex

    if (xMutex == NULL) {                // Failed to create mutex
        // ERROR HANDLING ROUTINE
    }
}
```

Create a Static Mutex: xSemaphoreCreateMutexStatic()

To create a mutex without using dynamic memory allocation, use the static version of the API.

`configSUPPORT_STATIC_ALLOCATION` and `configUSE_MUTEXES` must both be set to 1 in `FreeRTOSConfig.h`.

```
SemaphoreHandle_t xSemaphoreCreateMutexStatic( StaticSemaphore_t *pxMutexBuffer );
```

- **pxMutexBuffer:** A pointer to a variable of type `StaticSemaphore_t`, which will hold the mutex's control structure.
- **Return Value:** Returns a handle to the created mutex.
- **No Heap Usage:** Essential for safety-critical systems where deterministic memory usage is required.

Create a Static Mutex: xSemaphoreCreateMutexStatic()

```
#include "FreeRTOS.h"
#include "semphr.h"

StaticSemaphore_t xMutexBuffer;
SemaphoreHandle_t xMutex = NULL;

void vSetup(void) {
    xMutex = xSemaphoreCreateMutexStatic( &xMutexBuffer );    // Create the mutex

    // No need to check for NULL, as static allocation does not fail
}
```

Acquiring a Mutex: xSemaphoreTake()

The xSemaphoreTake() macro is used to "take" (lock) a mutex. A task must obtain the mutex before it can access the shared resource it protects.

```
#include "semphr.h"
```

```
BaseType_t xSemaphoreTake(SemaphoreHandle_t xSemaphore, TickType_t xTicksToWait);
```

- **xSemaphore:** Handle to the mutex being taken.
- **xTicksToWait:** The maximum time the task should remain in the Blocked state to wait for the mutex to become available.
- **Return Value:** Returns pdTRUE if the mutex was obtained, or pdFALSE if the timeout expired.

Mutex Waiting: Timeouts and Blocking

The `xTicksToWait` parameter defines how the task behaves if the mutex is already held by another task:

- **Zero Timeout (0):** The function returns immediately if the mutex is not available (polling).
- **Finite Timeout:** The task blocks for the specified number of ticks. If the mutex is released within this time, the task unblocks and takes it.
- **Indefinite Wait (`portMAX_DELAY`):** If `INCLUDE_vTaskSuspend` is set to 1, the task will block indefinitely until the mutex is released.

Releasing a Mutex: xSemaphoreGive()

The xSemaphoreGive() macro is used to "give" (unlock) a mutex that was previously taken. This makes the shared resource available to other tasks.

```
#include "semphr.h"
```

```
BaseType_t xSemaphoreGive(SemaphoreHandle_t xSemaphore);
```

- **Ownership:** A mutex **must only** be released by the same task that successfully took it.
- **Return Value:** Returns pdTRUE if the mutex was released successfully, or pdFALSE if an error occurred (e.g., the calling task is not the owner).
- **Unblocking:** If other tasks are blocked waiting for this mutex, giving it back will unblock the highest-priority task.

Example: Protecting a Shared Resource

```
void vTaskFunction(void *pvParameters) {
    if (xSemaphoreTake(xMutex, portMAX_DELAY) == pdFALSE) {    // Acquire mutex, unbounded blocking.
        printf("Failed to take mutex\n");                        // Handle error (mutex not acquired) and exit.
        return;
    }

    /* --- CRITICAL SECTION START --- */

    // Access shared resource (e.g., UART, Global Buffer)
    printf("Task accessing shared resource\n");

    /* --- CRITICAL SECTION END --- */

    xSemaphoreGive(xMutex);                                     // Release ownership
}
```

Example: Protecting a Shared Resource with Timeout

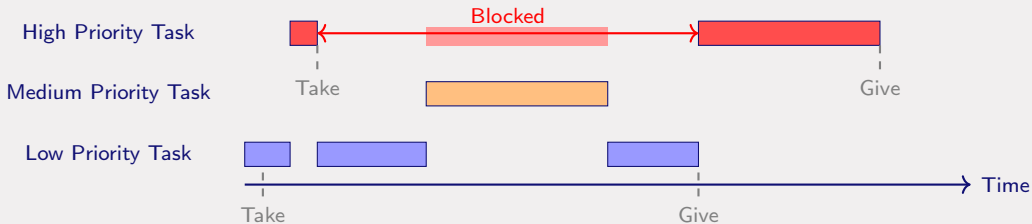
```
void vTaskCode(void *pvParameters) {
    for (;;) {
        if (xSemaphoreTake(xMutex, pdMS_TO_TICKS(100)) == pfFALSE) { // Take the mutex (wait up to 100ms)
            // Could not acquire resource - handle timeout
            printf("Failed to take mutex\n");
            // Skip this iteration and try again later
            continue;
        }

        /* Access the shared resource safely */

        xSemaphoreGive(xMutex); // Give the mutex back
    }
}
```

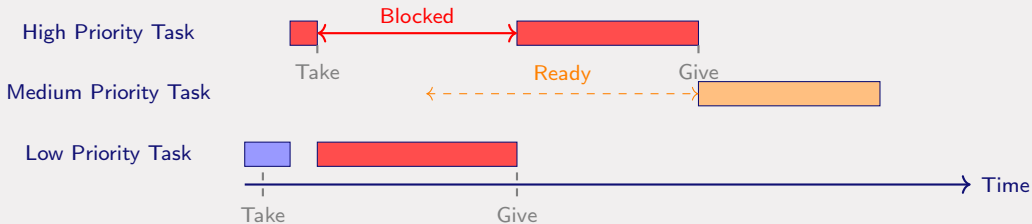
The Problem of Priority Inversion

1. **Low Priority (LPT)** takes a Mutex.
2. **High Priority (HPT)** tries to take the Mutex and blocks.
3. **Medium Priority (MPT)** wakes up and preempts LPT (because it doesn't need the mutex).
4. **Result:** HPT is waiting for LPT, but LPT cannot run because MPT is running. HPT is effectively delayed by MPT.



Bounding Blocking: Priority Inheritance

- **Mechanism:** If a High Priority Task blocks on a mutex held by a Low Priority Task, the LPT *temporarily* inherits the priority of the HPT.
- **Effect:** The LPT can now finish its work without being preempted by Medium Priority Tasks.
- **Cleanup:** Once the Mutex is released, the LPT returns to its original base priority.



Recursive Mutexes

- **Deadlock Scenario:** A task holding a mutex calls a function that tries to take the *same* mutex.
- **Recursive Mutex Solution:**
 - ▶ Allows the owner to “*re-take*” the mutex multiple times.
 - ▶ Must be “given” back the exact same number of times it was taken.
 - ▶ API: `xSemaphoreCreateRecursiveMutex()`, `xSemaphoreTakeRecursive()`.

Gatekeeper Tasks: A Clean Approach to Mutual Exclusion

Direct access to shared resources (UART, LCD, Sensors) can cause:

- **Priority Inversion:** High-priority tasks blocked by low-priority tasks.
- **Deadlocks:** Circular dependencies when locking mutexes.
- **ISR Complexity:** Difficulty accessing resources within interrupt routines.

A **Gatekeeper Task** is a dedicated task that serializes access to a shared resource, eliminating these issues.

What is a Gatekeeper Task?

- **Sole Ownership:** It is the only task permitted to access the resource directly.
- **Indirect Access:** Any other task requiring the resource must do so via the gatekeeper's services.
- **Serialization:** Uses queues to organise incoming requests.

Service Model

[Task A] → [Queue] → [**Gatekeeper**] → [Resource]

Advantages

1. **Zero Risk of Priority Inversion:** No task holds a mutex while being preempted.
2. **Deadlock Immunity:** No circular lock dependencies.
3. **Internal Simplicity:** The gatekeeper does not need to consider mutual exclusion within its own code (it is the sole owner).
4. **ISR Safety:** Interrupts can send data to queues safely.

Implementation: The Gatekeeper Task

```
void prvStdioGatekeeperTask( void *pvParameters ) {
    char *pcMessageToPrint;
    for( ;; ) {
        /* Wait indefinitely for a message to arrive in the queue */
        xQueueReceive( xPrintQueue, &pcMessageToPrint, portMAX_DELAY );

        /* Direct output to the resource (sole owner) */
        printf( "%s", pcMessageToPrint );
        fflush( stdout );

        /* Return to Blocked state to optimise CPU usage */
    }
}
```

The Client Side: Sending Requests

```
void prvPrintTask( void *pvParameters ) {  
    int iIndexToString = ( int ) pvParameters;  
    for( ;; ) {  
        /* Do not call printf directly! Send to the queue. */  
        xQueueSendToBack( xPrintQueue, &(amp; some_static_array[ iIndexToString ] ), 0 );  
  
        /* Wait for a pseudo-random duration */  
        vTaskDelay( ( rand() % 0x20 ) );  
    }  
}
```

Comparison: Mutex vs. Gatekeeper

Feature	Traditional (Mutex)	Gatekeeper Task
Priority Inversion	High Risk	Non-existent
Deadlock	Possible	Unlikely
ISR Access	Complex/Prohibited	Simple via Queues
Ownership	Dynamic (multiple tasks)	Static (one task)
Determinism	Variable (blocking)	High (serialised)

Architectural Considerations

- **Queue Depth:** Must be sufficient to handle bursts of messages from multiple tasks and interrupts.
- **Task Priority:** The gatekeeper's priority should reflect the resource's importance within the system.
- **Data Persistence:** When sending pointers through a queue, ensure the pointed data is not local (stack) memory that will expire.

Conclusion

- Gatekeeper tasks shift the paradigm from **"Resource Locking"** to **"Message Passing"**.
- They provide a robust architecture for handling output peripherals (UART, LCD, Logs).
- They eliminate the most common pitfalls in real-time systems (Priority Inversion and Deadlocks).

Mutexes vs. ISR

Never Use Mutexes in ISRs

Mutexes cannot be used inside an Interrupt Service Routine (ISR) because:

- They include a priority inheritance mechanism (which only makes sense for tasks).
- Taking a mutex can cause a block, which is illegal in ISRs.

Summary & Best Practices

- Use **Mutexes** for resource protection.
- Use **Semaphores** for task synchronisation.
- Keep the time a Mutex is held to a **minimum**.
- Always check the return value of `xSemaphoreTake()`.
- Design your system to avoid circular dependencies (deadlocks).